

What is Lambda Expression ?

It is a new feature introduced from JDK 1.8V.

* It is used to enable **functional programming** in Java.

* It is mainly used to concise our code (reduce the length of the code).

* A Lambda is a anonymous function (function/method without any name) which does not **need** :

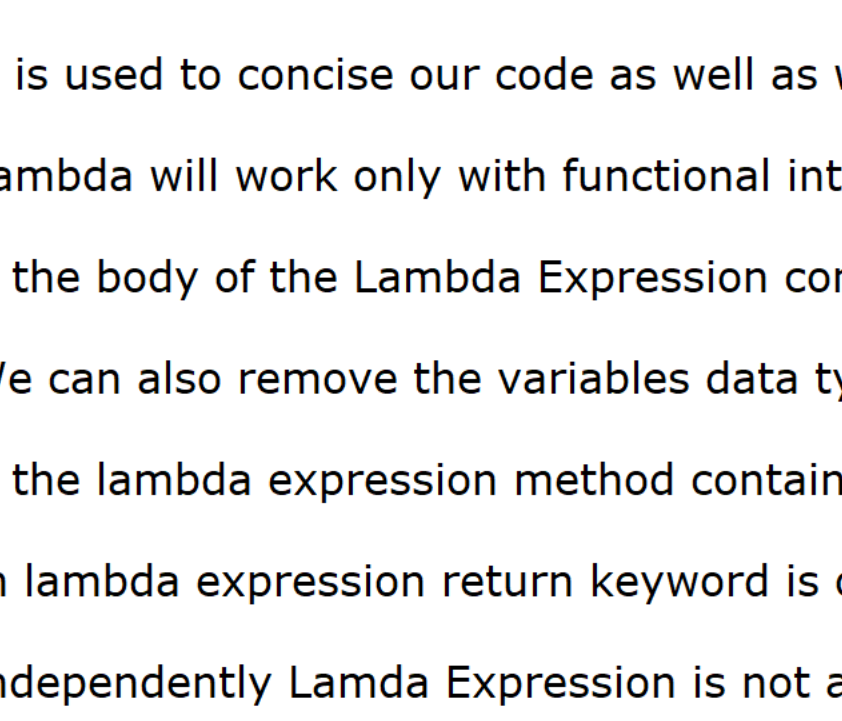
- a) Access modifier of the method not required
- b) Name of the method not required
- c) Return type of the method not required
- d) Parameter Variable type is also not required

Different cases for writing Lambda Expression :

Case 1:	
-----	-----
Concrete Method :	Lambda Expression :
-----	-----
public void display() { IO.println("display method"); }	() -> IO.println("display method");
Case 2 :	
-----	-----
Concrete method :	Lambda Expression :
-----	-----
public int sum(int x, int y) { IO.println("Sum is :"+(x+y)); }	(x, y) -> IO.println("Sum is :"+(x+y));
Case 3 :	
-----	-----
Concrete Method :	Lambda Expression :
-----	-----
public int findLength(String str) { return str.length(); }	str -> str.length();

Anonymous inner class VS Lambda Expression :

Anonymous Inner class	Lambda Expression
-----	-----
@FunctionalInterface public interface Printable { void print(); //SAM }	@FunctionalInterface public interface Printable { void print(); //SAM }
public class Main { public static void main(String [] args) { Printable p = new Printable() { @Override public void print() { IO.println("Printing"); } }; p.print(); } }	public class Main { public static void main(String [] args) { Printable p = () -> IO.println("Printing"); p.print(); } }


Boiler-plate code

It is used to concise our code as well as we can remove boilerplate code.

Lambda will work only with functional interface.

If the body of the Lambda Expression contains only one statement then curly braces are optional.

We can also remove the variables data type while defining the Lambda Expression parameter.

If the lambda expression method contains only one parameter then we can remove () symbol also.

In lambda expression return keyword is optional but if we use return keyword then {} are compulsory.

Independently Lamda Expression is not a statement.

Lambda target must be a functional interface only.Lamda target can't be class or abstract class, it will work with functional interface only.

While working with Lambda expression we can use class level (static + non static) and Method level variable (Local + parameter) but while using the lambda, our local variable must be final OR effectively final.

For Every Lambda JVM will generate a dynamic class at runtime to support Lambda using LambdaMetaFactory (Java 8) and this class internally implements from corresponding interface to override the abstract method.

```
//Programs :
-----
@FunctionalInterface
interface Printable
{
    void print();
}

public class LambdaDemo
{
    public static void main(String[] args)
    {
        Printable p = ()-> IO.println("Printing using Lambda");
        p.print();
        IO.println(p.getClass().getName());
    }
}
```

```
package com.ravi.lambda;

interface Vehicle
{
    void run();
}

public class LambdaDemo1
{
    public static void main(String[] args)
    {
        Vehicle car = ()-> IO.println("Car is running");
        car.run();

        Vehicle bus = ()-> IO.println("Bus is running");
        bus.run();
    }
}
```

```
package com.ravi.lambda;

interface Calculator
{
    void sum(int x, int y);
}

public class LambdaDemo2
{
    public static void main(String[] args)
    {
        Calculator c1 = (c, d)-> IO.println("Sum is :"+(c+d));
        c1.sum(12, 90);
    }
}
```

```
package com.ravi.lambda;

@FunctionalInterface
interface Length
{
    int getLength(String str);
}

public class LambdaDemo3
{
    public static void main(String[] args)
    {
        Length length = str->
        {
            return str.length();
        };
        IO.println(length.getLength("Ravi"));
    }
}
```

```
interface Callable
{
    public abstract void call();
}

public class LambdaDemo4
{
    public static void main(String[] args)
    {
        Callable c1 = ()-> IO.println("Calling");
        c1.call();
    }
}
```

```
package com.ravi.lambda;

@FunctionalInterface
interface Drawable
{
    void draw();
}

public class LambdaDemo5 {

    public static void main(String[] args)
    {
        String type = "Rectangle"; //must be final OR effectively final
        //type = "circle";

        Drawable d1 = ()-> IO.println("Drawing "+type);
        d1.draw();
    }
}
```